

Sprint 7. Creación de funciones, estructuras de datos y bucles

NIVEL 1

EJERCICIO 1

Esta función pide al usuario los datos de un nuevo candidato, devuelve en forma de diccionario con los datos de estos. He usado el parámetro `test_data=None` para que la función pueda funcionar en dos modos: si le pasas la lista de datos de prueba, usa esos datos automáticamente; si no le pasas nada (queda en `None`), pide los datos al usuario por teclado de manera interactiva.

```
[47]  
✓ 0 s  
  
import pandas as pd  
  
inputs_test = [  
    "Laura Serra", "608112233", 6, 3, 3, 3,  
    "Clara Gómez", "655221199", 3, 3, 3, 2,  
    "Sergio López", "644112211", 3, 2, 2, 2,  
    "Candidat Error", "611000000", 5, 7, 3, 2,  
    "Marta Ruiz", "699887766", 2, 3, 2, 1,  
    "Marc Vidal", "612443322", 6, 3, 3, 3,  
    "Júlia Romero", "677998877", 5, 3, 3, 2,  
    "Pol Navarro", "699554433", 4, 3, 2, 2,  
    "David Ferrer", "677665544", 2, 1, 1, 1,  
    "Anna Paredes", "633221144", 1, 2, 1, 0,  
    "Anibal Paredes", "644272727", 1, 2, 1, 0,  
    "Núria Casals", "600112200", 8, 2, 3, 3  
]  
  
def obtener_datos(index=0, test_data=None):  
    if test_data and index < len(test_data):  
        return {  
            "nombre": test_data[index],  
            "telefono": test_data[index + 1],  
            "años": test_data[index + 2],  
            "catalan": test_data[index + 3],  
            "excel": test_data[index + 4],  
            "titulo": test_data[index + 5],  
        },  
    }
```

```
[47]
✓ Os
def obtener_datos(index=0, test_data=None):
    if test_data and index < len(test_data):
        return {
            "nombre": test_data[index],
            "telefono": test_data[index + 1],
            "años": test_data[index + 2],
            "catalan": test_data[index + 3],
            "excel": test_data[index + 4],
            "titulo": test_data[index + 5],
        }
    print("Introducir datos del candidato:")
    return {
        "nombre": input("Nombre y apellidos: "),
        "telefono": input("Teléfono: "),
        "años": int(input("Años de experiencia: ")),
        "catalan": int(input("Catalán (0-3): ")),
        "excel": int(input("Excel (0-3): ")),
        "titulo": int(input("Título (0-3): ")),
    }

test_data = [
    "Sergio López", "644112211", 3, 2, 2, 2,
    "Candidat Error", "611000000", 5, 7, 3, 2,
    "Marta Ruiz", "699887766", 2, 3, 2, 1,
    "Marc Vidal", "612443322", 6, 3, 3, 3,
    "Júlia Romero", "677998877", 5, 3, 3, 2,
    "Pol Navarro", "699554433", 4, 3, 2, 2,
    "David Ferrer", "677665544", 2, 1, 1, 1,
    "Anna Paredes", "633221144", 1, 2, 1, 0,
    "Anibal Paredes", "64422727", 1, 2, 1, 0,
    "Núria Casals", "680112289", 8, 2, 3, 3
]
```

Esta función verifica que todos los valores numéricos estén dentro de los rangos permitidos. recibe los argumentos de datos diccionarios y devuelve un (True/False) y un mensaje de error.

```
[2]
✓ Os
def validar_datos(datos):
    """Valida rangos de los datos."""
    errores = []
    try:
        if not (0 <= datos["catalan"] <= 3):
            errores.append("Catalán fuera de rango (0-3)")

        if not (0 <= datos["excel"] <= 3):
            errores.append("Excel fuera de rango (0-3)")

        if not (0 <= datos["titulo"] <= 3):
            errores.append("Título fuera de rango (0-3)")

        if datos["años"] < 0:
            errores.append("Años negativos no válidos")

        if errores:
            return False, "\n".join(errores)

        return True, ""
    except Exception:
        return False, "Error en el formato de datos"
```

Esta función calcula la puntuación total del candidato (años * 1.5 + resto de puntos). Recibe los datos en forma de diccionario y devuelve una puntuación (float)

```
[1]
✓ Os
def calcular_puntuacion(datos):
    """Calcula la puntuación total."""
    return round(datos["años"] * 1.5 + datos["catalan"] + datos["excel"] + datos["titulo"], 2)
```

Esta función devuelve la categoría del candidato según los umbrales de puntuación. Recibe los argumentos de puntuación en float y devuelve una categoría en string.

```
[4]
✓ Os
def clasificar_candidato(puntuacion):
    """Clasifica al candidato según su puntuación."""
    if puntuacion >= 12:
        return "Prioridad Alta"
    elif puntuacion >= 8:
        return "Candidato adecuado"
    else:
        return "No prioritario"
```

Esta función recibe la variable candidato, puntuación y categoría y muestra en pantalla la información y resultado de un candidato individual.

```
[5]
✓ Os
def mostrar_individual(candidato, puntuacion, categoria):
    """Muestra resultado individual."""
    print(f"Candidato: {candidato['nombre']}")
    print(f"Teléfono: {candidato['telefono']}")
    print(f"Puntuación: {puntuacion} - {categoria}")
```

Esta función genera y muestra un informe final ordenado usando pandas DataFrame.

```
[6]
✓ Os
def mostrar_informe(candidatos):
    if not candidatos:
        print("No hay candidatos para mostrar.")
        return

    df = pd.DataFrame(candidatos)

    prioridad_orden = {"Prioridad Alta": 2, "Candidato adecuado": 1, "No prioritario": 0}
    df["prioridad_num"] = df["categoria"].map(prioridad_orden)
    df = df.sort_values(by=["prioridad_num", "puntuacion"], ascending=[False, False])
    df = df.drop(columns=["prioridad_num"])

    print(df[["nombre", "telefono", "puntuacion", "categoria"]].to_string(index=False))
```

Esta función orquesta todo el proceso: registra candidatos, valida y genera el informe final.

[53]
✓ 0 s

```
def principal_seleccion():
    candidatos = []
    for i in range(0, len(inputs_test), 6):
        if len(candidatos) >= 10:
            break

        datos = obtener_datos(i, inputs_test)
        valido, mensaje_error = validar_datos(datos)

        if valido:
            puntuacion = calcular_puntuacion(datos)
            categoria = clasificar_candidato(puntuacion)

            mostrar_individual(datos, puntuacion, categoria)

            candidatos.append({
                "nombre": datos["nombre"],
                "telefono": datos["telefono"],
                "puntuacion": puntuacion,
                "categoria": categoria
            })
        else:
            print(f"Error en {datos.get('nombre', 'candidato')}: {mensaje_error}")

    mostrar_informe(candidatos)

principal_seleccion()
```

```
*** Candidato: Laura Serra
    Teléfono: 608112233
    Puntuación: 18.0 → Prioridad Alta
    Candidato: Clara Gómez
    Teléfono: 655221199
    Puntuación: 12.5 → Prioridad Alta
    Candidato: Sergio López
    Teléfono: 644112211
    Puntuación: 10.5 → Candidato adecuado
    Error en Candidat Error: Catalán fuera de rango (0-3)
    Candidato: Marta Ruiz
    Teléfono: 699887766
    Puntuación: 9.0 → Candidato adecuado
    Candidato: Marc Vidal
    Teléfono: 612443322
    Puntuación: 18.0 → Prioridad Alta
    Candidato: Júlia Romero
    Teléfono: 677998877
    Puntuación: 15.5 → Prioridad Alta
    Candidato: Pol Navarro
    Teléfono: 699554433
    Puntuación: 13.0 → Prioridad Alta
    Candidato: David Ferrer
    Teléfono: 677665544
    Puntuación: 6.0 → No prioritario
    Candidato: Anna Paredes
    Teléfono: 633221144
    Puntuación: 4.5 → No prioritario
    Candidato: Anibal Paredes
    Teléfono: 644272727
    Puntuación: 4.5 → No prioritario
```

Puntuación: 4.5 - No prioritario

nombre	telefono	puntuacion	categoria
Laura Serra	608112233	18.0	Prioridad Alta
Marc Vidal	612443322	18.0	Prioridad Alta
Júlia Romero	677998877	15.5	Prioridad Alta
Pol Navarro	699554433	13.0	Prioridad Alta
Clara Gómez	655221199	12.5	Prioridad Alta
Sergio López	644112211	10.5	Candidato adecuado
Marta Ruiz	699887766	9.0	Candidato adecuado
David Ferrer	677665544	6.0	No prioritario
Anna Paredes	633221144	4.5	No prioritario
Anibal Paredes	644272727	4.5	No prioritario

EJERCICIO 2

Esta función obtiene el factor multiplicador según el grupo de edad. Recibe el grupo_edad (string) y Devuelve factor numérico

```
[30]
✓ 0 s
import pandas as pd

clientes = [
    {"id_cliente": 1, "nombre": "Laura Serra", "edad": 23, "grupo_edad": "18-25", "codigo_comportamiento": 4},
    {"id_cliente": 2, "nombre": "Marc Vidal", "edad": 35, "grupo_edad": "26-40", "codigo_comportamiento": 3},
    {"id_cliente": 3, "nombre": "Ana López", "edad": 29, "grupo_edad": "26-40", "codigo_comportamiento": 2},
    {"id_cliente": 4, "nombre": "Joan Riera", "edad": 19, "grupo_edad": "18-25", "codigo_comportamiento": 2},
    {"id_cliente": 5, "nombre": "Mónica Pérez", "edad": 47, "grupo_edad": "41-60", "codigo_comportamiento": 4},
    {"id_cliente": 6, "nombre": "Luis García", "edad": 52, "grupo_edad": "41-60", "codigo_comportamiento": 1},
    {"id_cliente": 7, "nombre": "Pilar Sánchez", "edad": 61, "grupo_edad": "60+", "codigo_comportamiento": 3},
    {"id_cliente": 8, "nombre": "Eva Martín", "edad": 38, "grupo_edad": "26-40", "codigo_comportamiento": 0},
    {"id_cliente": 9, "nombre": "Diego Romero", "edad": 24, "grupo_edad": "18-25", "codigo_comportamiento": 1},
    {"id_cliente": 10, "nombre": "Núria Costa", "edad": 33, "grupo_edad": "26-40", "codigo_comportamiento": 4}
]

factores_comportamiento = {0: 0.10, 1: 0.25, 2: 0.50, 3: 0.75, 4: 0.90}
factores_edad = {"18-25": 1.20, "26-40": 1.00, "41-60": 0.85, "60+": 0.70}

def obtener_factor_edad(grupo):
    return factores_edad.get(grupo, 1.0)
```

Esta función devuelve el segmento y mensaje según código de comportamiento. Recibe el código de comportamiento en int y devuelve un factor numérico.

```
[31]
✓ 0 s
def obtener_segmento_y_factor(codigo):

    segmentos = {
        0: ("Bajo interés", "¡Explora más ofertas!"),
        1: ("Interés moderado", "¡Recuerda tus productos!"),
        2: ("A punto de comprar", "¡Finaliza la compra ahora!"),
        3: ("Abandono reciente", "¡Vuelve a tu carrito!"),
        4: ("Abandono antiguo", "¡No pierdas tus ofertas!")
    }

    return segmentos.get(codigo, ("Desconocido", "Contáctanos"))
```

Esta función calcula el riesgo final multiplicando ambos factores. Recibe código, grupo_edad y devuelve riesgo_final (float).

```
[32]
✓ 0 s
def calcular_riesgo(codigo, grupo_edad):
    return factores_comportamiento.get(codigo, 0) * obtener_factor_edad(grupo_edad)
```

Esta función procesa la lista de clientes añadiendo los campos calculados.

```
[33]
✓ 0s
def procesar_clientes(clientes):
    for cliente in clientes:
        cliente["factor_codigo"] = factores_comportamiento.get(cliente["codigo_comportamiento"], 0)
        cliente["factor_edad"] = obtener_factor_edad(cliente["grupo_edad"])
        cliente["riesgo_final"] = calcular_riesgo(cliente["codigo_comportamiento"], cliente["grupo_edad"])

        segmento, mensaje = obtener_segmento_y_factor(cliente["codigo_comportamiento"])
        cliente["segmento"] = segmento
        cliente["mensaje"] = mensaje
    return clientes
```

Esta función muestra los clientes procesados como DataFrame

```
[34]
✓ 0s
def mostrar_clientes(procesados):
    df = pd.DataFrame(procesados)
    print(df[["id_cliente", "nombre", "edad", "grupo_edad", "codigo_comportamiento",
              "segmento", "riesgo_final"]].round(2).to_string(index=False))
    return df
```

Esta función responderá las 4 preguntas del departamento de Marketing. Primero se mostrará los clientes más urgentes para contactar, después filtrando el cliente por código de comportamiento mostraremos el riesgo según la edad, seguidamente mostraremos los clientes con un comportamiento 0 y finalmente encontraremos al cliente con mayor riesgo.

```
[35]
✓ 0s
def mostrar_respuestas(procesados):

    ordenados_riesgo = sorted(procesados, key=lambda x: x['riesgo_final'], reverse=True)

    print("\n=== RESPUESTAS DE MARKETING ===")

    print("1. Clientes a contactar primero en campaña de retargeting:")
    for cliente in ordenados_riesgo[:3]:
        print(f"    → {cliente['nombre']} (Riesgo: {cliente['riesgo_final']:.2f}) - {cliente['mensaje']}")

    print("\n2. Diferencias por edad:")
    clientes_codigo4 = [cliente for cliente in procesados if cliente['codigo_comportamiento'] == 4]
    for cliente in clientes_codigo4:
        print(f"    → {cliente['nombre']} ({cliente['grupo_edad']}) → Riesgo = {cliente['riesgo_final']:.2f}")

    clientes_bajo_interes = [cliente for cliente in procesados if cliente['codigo_comportamiento'] == 0]
    print("\n3. Clientes con tan poco interés que no vale la pena invertir:")
    for cliente in clientes_bajo_interes:
        print(f"    → {cliente['nombre']} (Código 0 - Riesgo: {cliente['riesgo_final']:.2f})")

    cliente_mas_rendible = max(procesados, key=lambda x: x['riesgo_final'])
    print(f"\n4. Cliente más rendible de recuperar: {(cliente_mas_rendible['nombre'])} ** "
          f"(Riesgo: {cliente_mas_rendible['riesgo_final']:.2f})")
```

Esta es la función principal que orquesta todo el análisis de segmentación

```
[42]
✓ 0s
def main_marketing():

    print("Iniciando análisis de segmentación por riesgo de cesta de compra...\n")

    clientes_procesados = procesar_clientes(clientes)
    mostrar_clientes(clientes_procesados)
    mostrar_respuestas(clientes_procesados)

    main_marketing()

*** Iniciando análisis de segmentación por riesgo de cesta de compra...

   id_cliente  nombre  edad grupo_edad  codigo_comportamiento  segmento  riesgo_final
1          1  Laura Serra    23    18-25                4  Abandono antiguo         1.08
2          2  Marc Vidal    35    26-40                3  Abandono reciente         0.75
3          3  Ana López    29    26-40                2  A punto de comprar         0.50
4          4  Joan Riera    19    18-25                2  A punto de comprar         0.60
5          5  Mónica Pérez   47    41-60                4  Abandono antiguo         0.76
6          6  Luis García    52    41-60                1  Interés moderado         0.21
7          7  Pilar Sánchez   61    60+                3  Abandono reciente         0.52
8          8  Eva Martín    38    26-40                0  Bajo interés         0.10
9          9  Diego Romero   24    18-25                1  Interés moderado         0.30
10         10  Núria Costa    33    26-40                4  Abandono antiguo         0.90

=== RESPUESTAS DE MARKETING ===
1. Clientes a contactar primero en campaña de retargeting:
  → Laura Serra (Riesgo: 1.08) - ¡No pierdas tus ofertas!
  → Núria Costa (Riesgo: 0.90) - ¡No pierdas tus ofertas!
  → Mónica Pérez (Riesgo: 0.77) - ¡No pierdas tus ofertas!
```

[42]

✓ 0 s



main_marketing()



... Iniciando análisis de segmentación por riesgo de cesta de compra...

id_cliente	nombre	edad	grupo_edad	codigo_comportamiento	segmento	riesgo_final
1	Laura Serra	23	18-25	4	Abandono antiguo	1.08
2	Marc Vidal	35	26-40	3	Abandono reciente	0.75
3	Ana López	29	26-40	2	A punto de comprar	0.50
4	Joan Riera	19	18-25	2	A punto de comprar	0.60
5	Mónica Pérez	47	41-60	4	Abandono antiguo	0.76
6	Luis García	52	41-60	1	Interés moderado	0.21
7	Pilar Sánchez	61	60+	3	Abandono reciente	0.52
8	Eva Martín	38	26-40	0	Bajo interés	0.10
9	Diego Romero	24	18-25	1	Interés moderado	0.30
10	Núria Costa	33	26-40	4	Abandono antiguo	0.90

=== RESPUESTAS DE MARKETING ===

1. Clientes a contactar primero en campaña de retargeting:
→ Laura Serra (Riesgo: 1.08) - ¡No pierdas tus ofertas!
→ Núria Costa (Riesgo: 0.90) - ¡No pierdas tus ofertas!
→ Mónica Pérez (Riesgo: 0.77) - ¡No pierdas tus ofertas!

2. Diferencias por edad :
→ Laura Serra (18-25) → Riesgo = 1.08
→ Mónica Pérez (41-60) → Riesgo = 0.77
→ Núria Costa (26-40) → Riesgo = 0.90

3. Clientes con tan poco interés que no vale la pena invertir:
→ Eva Martín (Código 0 - Riesgo: 0.10)

4. Cliente más rentable de recuperar: **Laura Serra** (Riesgo: 1.08)

NIVEL 2

EJERCICIO 1

Nivel 2 Ejercicio 2

Limpia y normaliza cada sintoma (minúsculas, sin espacios, etc)

```
[54]
✓ 0s
import pandas as pd

pacientes = [
    "Maria|45|tos, Fiebre Alta, dolor_pecho",
    "Luis|33|dolor_cabeza,fatiga",
    "Sara|67|fiebre alta, dificultad RESPIRAR, tos",
    "Jordi|52|fatiga , tos",
    "Anna|29|fiebre alta,dolor_cabeza"
]

niveles_riesgo = {
    "fiebre_alta": 3,
    "dificultad_respirar": 5,
    "dolor_pecho": 4,
    "tos": 1,
    "fatiga": 1,
    "dolor_cabeza": 1
}

def limpiar_sintoma(sintoma):
    return sintoma.strip().lower().replace(" ", "_").replace("á", "a").replace("é", "e")
```

Separa una línea de texto en nombre, edad y lista de síntomas

```
[55]
✓ 0s
def separar_paciente(linea):
    partes = linea.strip().split("|")
    nombre = partes[0].strip()
    edad = int(partes[1].strip())
    sintomas_raw = [limpiar_sintoma(s) for s in partes[2].split(",")]
    return {"nombre": nombre, "edad": edad, "sintomas": sintomas_raw}
```

Calcula el riesgo total sumando los valores de cada sintoma.

```
[56]
✓ 0s
def calcular_riesgo_paciente(paciente, niveles_riesgo):
    riesgo_total = 0
    for sintoma in paciente["sintomas"]:
        riesgo_total += niveles_riesgo.get(sintoma, 0)
    paciente["riesgo_total"] = riesgo_total
    paciente["sintomas_normalizados"] = paciente["sintomas"]
    return paciente
```

Procesa todos los pacientes: separa datos, calcula riesgo y devuelve la lista completa.

```
[57]
✓ 0s
def procesar_pacientes(pacientes_raw, niveles_riesgo):
    procesados = []
    for linea in pacientes_raw:
        p = separar_paciente(linea)
        procesados.append(calcular_riesgo_paciente(p, niveles_riesgo))
    return procesados
```

Muestra el informe final ordenado por riesgo + Función principal que ejecuta todo el proceso.

```
[58]
✓ 0s
def mostrar_informe(pacientes_procesados):
    if not pacientes_procesados:
        print("No hay pacientes para mostrar.")
        return
    df = pd.DataFrame(pacientes_procesados)
    df = df.sort_values(by="riesgo_total", ascending=False)

    print("\n=== PACIENTES ORDENADOS POR PRIORIDAD (mayor riesgo primero) ===")
    print(df[["nombre", "edad", "riesgo_total", "sintomas_normalizados"]].to_string(index=False))

    # Ejecución
    def main_pacientes():
        print("Datos de pacientes\n")
        pacientes_procesados = procesar_pacientes(pacientes, niveles_riesgo)
        mostrar_informe(pacientes_procesados)

    main_pacientes()
```

▼ Datos de pacientes

```
=== PACIENTES ORDENADOS POR PRIORIDAD (mayor riesgo primero) ===
nombre edad riesgo_total sintomas_normalizados
Sara    67          9 [fiebre_alta, dificultad_respirar, tos]
Maria   45          8 [tos, fiebre_alta, dolor_pecho]
Anna    29          4 [fiebre_alta, dolor_cabeza]
Luis    33          2 [dolor_cabeza, fatiga]
Jordi   52          2 [fatiga, tos]
```

EJERCICIO 2

NIVEL 2 EJERCICIO 2

Procesa un cliente añadiendo categoría, precios y probabilidad de fraude.

[61]
✓ 0s

```
import pandas as pd

clientes = [
    {"nom": "Ana", "edat": 42, "codi": 0},
    {"nom": "Carlos", "edat": 30, "codi": 2},
    {"nom": "Isabel", "edat": 55, "codi": 3},
    {"nom": "Jorge", "edat": 40, "codi": 1},
    {"nom": "Marta", "edat": 28, "codi": 0}
]

niveles_seguro = {
    0: {"categoria": "Bajo riesgo", "precio_base": 120, "fraude": 0.01},
    1: {"categoria": "Riesgo medio", "precio_base": 200, "fraude": 0.05},
    2: {"categoria": "Riesgo alto", "precio_base": 350, "fraude": 0.15},
    3: {"categoria": "Riesgo critico", "precio_base": 500, "fraude": 0.30}
}

def procesar_cliente(cliente):
    info = niveles_seguro.get(cliente["codi"], {})
    cliente["categoria"] = info.get("categoria", "Desconocido")
    cliente["precio_base"] = info.get("precio_base", 0)
    cliente["prob_fraude"] = info.get("fraude", 0)
    cliente["precio_final"] = round(cliente["precio_base"] * (1 + cliente["edat"]/100), 2)
    return cliente
```

[62]
✓ 0s

```
def mostrar_clientes(procesados):

    df = pd.DataFrame(procesados)
    print("\n=== CLIENTES DE SEGUROS ===")
    print(df[["nom", "edat", "categoria", "precio_base", "precio_final", "prob_fraude"]].to_string(index=False))
    return df
```

Muestra todos los clientes procesados usando un DataFrame.

[63]
✓ 0s

```
def mostrar_analisis(procesados):
    ordenados_fraude = sorted(procesados, key=lambda x: x["prob_fraude"], reverse=True)
    mas_caro = max(procesados, key=lambda x: x["precio_final"])

    print("\n=== ANÁLISIS ===")
    print(f"Cliente con mayor probabilidad de fraude: {ordenados_fraude[0]['nom']} ({ordenados_fraude[0]['prob_fraude']})")
    print(f"Cliente con precio final más alto: {mas_caro['nom']} ({mas_caro['precio_final']})")
    print("Clientes prioritarios para revisión manual:", [cliente['nom'] for cliente in ordenados_fraude[:2]])
```

Muestra el análisis final con las conclusiones de marketing.

[64]
✓ 0s

```
def main_seguros():

    print("Clasificación de clientes de seguros.\n")

    # Procesar todos los clientes
    procesados = [procesar_cliente(c.copy()) for c in clientes]

    # Mostrar tabla y análisis
    mostrar_clientes(procesados)
    mostrar_analisis(procesados)

    main_seguros()
```

▼

*** Clasificación de clientes de seguros.

```
=== CLIENTES DE SEGUROS ===
 nom  edat  categoria  precio_base  precio_final  prob_fraude
Ana   42   Bajo riesgo      120         170.4         0.01
Carlos 30   Riesgo alto     350         455.0         0.15
Isabel 55  Riesgo critico    500         775.0         0.30
Jorge  40   Riesgo medio     200         280.0         0.05
Marta  28   Bajo riesgo      120         153.6         0.01

=== ANÁLISIS ===
Cliente con mayor probabilidad de fraude: Isabel (0.3)
Cliente con precio final más alto: Isabel (775.0)
Clientes prioritarios para revisión manual: ['Isabel', 'Carlos']
```


NIVEL 3

EJERCICIO 1

NIVEL 3 EJERCICIO 1

En esta función se seleccionan los caracteres :mayúsculas, minúsculas, números y signos. Se garantiza que la contraseña contenga al menos un carácter de cada tipo elegido, luego se completan los caracteres restantes de forma aleatoria hasta alcanzar la longitud marcada, se mezclan todos y finalmente se devuelve la contraseña generada.

```
[68]
✓ 0 s
import random
import string
def crear_contraseña(longitud=12, mayusculas=True, minusculas=True, numeros=True, signos=False):

    may = string.ascii_uppercase if mayusculas else ""
    min = string.ascii_lowercase if minusculas else ""
    num = string.digits if numeros else ""
    sig = string.punctuation if signos else ""

    caracteres_disponibles = may + min + num + sig

    if not caracteres_disponibles:
        return "Error: selecciona al menos un tipo de carácter"

    contraseña = []

    if mayusculas:
        contraseña.append(random.choice(may))
    if minusculas:
        contraseña.append(random.choice(min))
    if numeros:
```

```
import random
import string
def crear_contraseña(longitud=12, mayusculas=True, minusculas=True, numeros=True, signos=False):

    may = string.ascii_uppercase if mayusculas else ""
    min = string.ascii_lowercase if minusculas else ""
    num = string.digits if numeros else ""
    sig = string.punctuation if signos else ""

    caracteres_disponibles = may + min + num + sig

    if not caracteres_disponibles:
        return "Error: selecciona al menos un tipo de carácter"

    contraseña = []

    if mayusculas:
        contraseña.append(random.choice(may))
    if minusculas:
        contraseña.append(random.choice(min))
    if numeros:
        contraseña.append(random.choice(num))
    if signos:
        contraseña.append(random.choice(sig))

    while len(contraseña) < longitud:
        contraseña.append(random.choice(caracteres_disponibles))

    random.shuffle(contraseña)

    return ''.join(contraseña)

def main_generador():
```

```
(se)
✓ Os
▶ if mayusculas:
    contrasena.append(random.choice(may))
if minusculas:
    contrasena.append(random.choice(min))
if numeros:
    contrasena.append(random.choice(num))
if signos:
    contrasena.append(random.choice(sig))

while len(contrasena) < longitud:
    contrasena.append(random.choice(caracteres_disponibles))

random.shuffle(contrasena)

return ''.join(contrasena)

def main_generador():

    print("=== GENERADOR DE CONTRASEÑAS SEGURAS ===\n")

    print("Contraseña generada (12 caracteres):", crear_contrasena(12, signos=True))
    print("Ejemplo fuerte (16 caracteres):", crear_contrasena(16, mayusculas=True, minusculas=True, numeros=True, signos=True))

main_generador()

▼ ... === GENERADOR DE CONTRASEÑAS SEGURAS ===

Contraseña generada (12 caracteres): wsuYY~5Fpyo3
Ejemplo fuerte (16 caracteres): t~%4Vx2X;dlw!{
```

EJERCICIO 2